MyCodeSchool

Greșeli comune Spring — anti-pattern catalog

Lista celor mai frecvente greșeli pe care le văd reviewerii într-un PR Spring. Pentru fiecare: cod greșit, cod corect, **de ce**.

Folosește ca check înainte să trimiți cod la review.

---

1. Field injection cu `@Autowired`

❌

```
@Service
public class UserService {
    @Autowired
    private UserRepository repo;
}
```

✅

```
@Service
public class UserService {
    private final UserRepository repo;

    public UserService(UserRepository repo) {
        this.repo = repo;
    }
}
```

Why: field injection ascunde dependențele, împiedică `final`, blochează testul fără Spring (`new UserService(...)` imposibil).

---

2. `new Foo()` în loc de injecție

❌

```
@Service
public class OrderService {
    private final EmailSender sender = new EmailSender();  // bypass Spring
}
```

✅

```
@Service
public class OrderService {
    private final EmailSender sender;

    public OrderService(EmailSender sender) {
        this.sender = sender;
    }
}
```

Why: dacă instanțezi manual, Spring nu mai poate înlocui `EmailSender` cu un mock în teste, nu mai propagă proxy-uri (`@Transactional`, `@Async`), nu mai injectează *lui* dependențele.

---

3. State mutabil în bean singleton fără sincronizare

❌

```
@Service
public class CounterService {
    private int count = 0;          // shared mutable state

    public void increment() {
        count++;                    // race condition în multi-threading
    }
}
```

✅

```
@Service
public class CounterService {
    private final AtomicInteger count = new AtomicInteger();

    public void increment() {
        count.incrementAndGet();
    }
}
```

Why: bean-urile singleton sunt accesate de mai multe thread-uri simultan (request-uri web). Câmpurile mutabile fără sincronizare = bug-uri intermitente.

---

4. `@Value` pe câmp `static`

❌

```
@Component
public class Config {
    @Value("${app.api.key}")
    private static String apiKey;   // niciodată injectat
}
```

✅

```
@Component
public class Config {
    private final String apiKey;

    public Config(@Value("${app.api.key}") String apiKey) {
        this.apiKey = apiKey;
    }
}
```

Why: Spring injectează doar în câmpuri *instance*. Câmpurile `static` aparțin clasei, nu unui obiect — `@Value` e ignorat în tăcere, valoarea rămâne `null`.

---

5. Apel direct între metode `@Bean` în `@Component` (nu `@Configuration`)

❌

```
@Component                          // ar trebui @Configuration
public class AppConfig {
    @Bean
    public Random random() { return new Random(); }

    @Bean
    public RandomConsumer consumer() {
        return new RandomConsumer(random());  // creează un AL DOILEA Random
    }
}
```

✅

```
@Configuration                      // proxy CGLIB garantează singleton
public class AppConfig {
    @Bean
    public Random random() { return new Random(); }

    @Bean
    public RandomConsumer consumer() {
        return new RandomConsumer(random());  // returnează același Random
    }
}
```

Why: `@Configuration` creează proxy-uri care cache-uiesc apelurile între `@Bean`-uri. `@Component` nu face asta — fiecare apel direct e un apel Java normal, plus o instanță nouă.

---

6. `@PostConstruct` cu `javax.annotation.*` în Spring Boot 3

❌

```
// Spring Boot 3 = Jakarta, nu javax
import javax.annotation.PostConstruct;

@Component
public class CacheLoader {
    @PostConstruct
    void load() { ... }   // nu compilează SAU nu rulează
}
```

✅

```
import jakarta.annotation.PostConstruct;

@Component
public class CacheLoader {
    @PostConstruct
    void load() { ... }
}
```

Why: Spring Boot 3 a migrat la namespace-ul `jakarta.*`. Pachetul `javax.annotation` nu e pe classpath default.

---

7. Două bean-uri `@Primary` de același tip

❌

```
@Component @Primary class StripeGateway implements PaymentGateway { ... }
@Component @Primary class PaypalGateway implements PaymentGateway { ... }
```

✅

```
@Component @Primary class StripeGateway implements PaymentGateway { ... }
@Component          class PaypalGateway implements PaymentGateway { ... }
```

Why: `@Primary` înseamnă *unic default*. Două bean-uri `@Primary` recreează aceeași ambiguitate pe care ar trebui s-o rezolve → `NoUniqueBeanDefinitionException`.

---

8. Setter injection cu defaults `null`

❌

```
@Service
public class OrderService {
    private EmailSender sender;   // null până nu vine Spring

    @Autowired
    public void setSender(EmailSender sender) {
        this.sender = sender;
    }
}
```

✅

```
@Service
public class OrderService {
    private final EmailSender sender;

    public OrderService(EmailSender sender) {
        this.sender = sender;
    }
}
```

Why: setter injection lasă obiectul într-o stare incompletă între new și setter call. Constructor injection garantează că obiectul e *complet construit* înainte să fie folosit.

---

9. Catch generic `Exception` într-o metodă `@Bean`

❌

```
@Bean
public DataSource dataSource() {
    try {
        return buildDataSource();
    } catch (Exception e) {
        return null;                // ascunde config greșit
    }
}
```

✅

```
@Bean
public DataSource dataSource() {
    // lasă Spring să crash-eze cu mesaj clar
    return buildDataSource();
}
```

Why: dacă `@Bean` aruncă, Spring oprește bootstrap-ul cu mesaj precis. Dacă întorci `null`, eroarea apare mult mai târziu (NPE într-un consumator), departe de cauza reală.

---

10. `@Component` pe o clasă `@Configuration`

❌

```
@Component
@Configuration
public class AppConfig {
    @Bean public Random random() { ... }
}
```

✅

```
@Configuration
public class AppConfig {
    @Bean public Random random() { ... }
}
```

Why: `@Configuration` *deja conține* `@Component` (e meta-adnotat). Dublarea nu strică, dar e zgomot — confundă cititorul, sugerează că ar avea efecte distincte.

---

11. Folosirea `if (bean != null)` în loc să tratezi `NoSuchBean`

❌

```
@Service
public class OrderService {
    private final NotificationSender sender;   // poate să nu existe

    public OrderService(NotificationSender sender) {
        this.sender = sender;
    }
}
// Apoi sender == null check pe-undeva.
// Nu se întâmplă — Spring n-ar fi pornit.
```

✅ — dacă chiar e opțional, fă-o explicit:

```
public OrderService(ObjectProvider<NotificationSender> senderProvider) {
    this.sender = senderProvider.getIfAvailable();
}
```

Why: într-un constructor normal `@Autowired`, Spring eșuează la pornire dacă bean-ul lipsește. `null` nu apare. Defensive checks pe `null` ascund altceva (cum ar fi field injection cu setter chemat manual).

---

12. `@Lazy` aruncat peste tot ca să “rezolve” probleme

❌

```
@Component
public class Hub {
    @Lazy
    public Hub(@Lazy ServiceA a, @Lazy ServiceB b, @Lazy ServiceC c) { ... }
}
```

✅

```
@Component
public class Hub {
    public Hub(ServiceA a, ServiceB b, ServiceC c) { ... }
}
```

Why: `@Lazy` masquează design issues (ex. cicluri, sau wiring greoi). Dacă-ți trebuie pe tot constructorul, refactor — nu `@Lazy`-fy.

---

13. Hardcoding de valori care ar trebui în config

❌

```
@Service
public class PaymentService {
    // hardcoded
    private static final String API_URL = "https://api.stripe.com";
}
```

✅

```
@Service
public class PaymentService {
    private final String apiUrl;

    public PaymentService(@Value("${payment.api-url}") String apiUrl) {
        this.apiUrl = apiUrl;
    }
}
```

Why: valorile diferă între dev/test/prod. Hardcoded înseamnă recompilare la fiecare schimbare. `@Value` + `application.yaml` + profiles = config separat de cod.

---

14. `@SpringBootApplication` într-un sub-pachet

❌

```
src/main/java/
└── com/example/app/
    └── controllers/
        └── ApiApplication.java   // @SpringBootApplication aici
```

Apoi `com.example.app.services.MyService` (un @Component) nu e descoperit.

✅

```
src/main/java/
└── com/example/app/
    ├── ApiApplication.java       // @SpringBootApplication la rădăcină
    ├── controllers/
    └── services/
```

Why: `@SpringBootApplication` aduce cu el `@ComponentScan` care scanează pachetul propriu + sub-pachete. Dacă o pui într-un sub-pachet, fratele lui (`services/`) e în afara scanului.

15. `@Transactional` pe metodă privată sau pe constructor

❌

```
@Service
public class OrderService {
    @Transactional
    private void save(Order o) { ... }   // niciodată într-o tranzacție
}
```

✅

```
@Service
public class OrderService {
    @Transactional
    public void save(Order o) { ... }
}
```

Why: `@Transactional` funcționează prin proxy. Proxy-ul Spring interceptează doar apeluri *publice externe*. Metodele private sau apelurile din *aceeași clasă* prin `this.save(...)` ocolesc proxy-ul → fără tranzacție. (Vezi în L05+ când ajungem la `@Transactional` în detaliu — pune-l doar pe `public` + apelează din *altă* clasă.)

Cum folosești lista

- Înainte de PR: scanează lista repede, vezi dacă recunoști vreuna în codul tău.
- La code review: dacă vezi una, link-uiește către pct. respectiv.
- Discord/Slack al echipei: ține-o salvată, e cea mai bună documentație de onboarding.